

Tugas Kelompok Berbasis Kasus 01

Robot Kurir Kampus: Rute GU (Gerbang Utama) ke LK (Lab Komputasi). Metode: UCS, IDS, GBFS, dan A Star. Inteligensi Buatan, 2026.

Kelompok: Rx. Nama/NIM: [isi nim1, nim2, nim3]. Draft otomatis dari `search_kampus.py`.

Deskripsi Kasus

Kasus 01 bercerita tentang robot kurir kampus yang diberi misi mengantarkan barang dari Gerbang Utama yang disingkat GU menuju Lab Komputasi yang disingkat LK dengan melewati peta kampus yang dimodelkan sebagai graf tak berarah dan berbobot. Peta tersebut terdiri dari dua belas lokasi yaitu Gerbang Utama, Parkir Barat, Rektorat, Gedung Kuliah Umum, Perpustakaan, Kantin, Masjid, Aula, Asrama, Teknik Informatika, Sport Center, dan Lab Komputasi yang saling terhubung melalui sembilan belas jalur dengan bobot yang berbeda beda yang menyatakan jarak tempuh antar lokasi. Setiap lokasi juga dilengkapi dengan nilai heuristik h dalam kurung n yang berfungsi sebagai perkiraan jarak dari lokasi tersebut ke LK sehingga robot tidak berjalan secara membabi buta. Untuk menyelesaikan misi ini robot diuji dengan empat algoritma pencarian yang memiliki karakter berbeda beda yaitu UCS yang selalu memperluas simpul dengan biaya perjalanan terkecil dari titik awal, IDS yang melakukan pencarian mendalam berulang dengan batas kedalaman yang dinaikkan secara bertahap dan memakai bobot graf untuk menghitung total biaya di akhir, GBFS yang bersifat rakus karena hanya mengikuti nilai heuristik terkecil, serta A Star yang menggabungkan biaya perjalanan dan heuristik dalam fungsi f yang dihitung dari g ditambah h . Seluruh proses tersebut diwujudkan dalam sebuah program sederhana agar urutan simpul yang diperluas, rute yang ditemukan, dan total biayanya tercatat secara objektif. Hasil luaran program inilah yang kemudian dibandingkan dan dianalisis untuk melihat algoritma mana yang paling optimal, mana yang paling hemat langkah, dan mengapa perbedaan itu bisa terjadi pada peta yang sama.

Simpul	Tetangga (bobot)
GU	PB(3), R(4)
PB	GU(3), GKU(4), K(6)
R	GU(4), PR(3), M(5)
GBKU	PB(4), PR(2), K(3), IF(7)
K	PB(6), GKB(3), AS(5)
PR	R(3), GKB(2), A(6), IF(4)
A	PR(6), M(3), LK(5)
M	R(5), A(3), SC(6)
IF	GBKU(7), PR(4), AS(4), LK(3)
AS	K(5), IF(4), SC(6)
SC	M(6), AS(6), LK(4)
LK	A(5), IF(3), SC(4)

Simpul	$h(n)$	Simpul	$h(n)$	Simpul	$h(n)$
GU	12	PB	10	R	9
GBKU	7	PR	6	K	9
M	7	A	4	AS	5
IF	2	SC	3	LK	0

Heuristik ini admissible dan konsisten. Contoh: $R(9) \leq 3 + 6 = 9$ dan $PR(6) \leq 4 + 2 = 6$, sehingga A Star dijamin optimal di graf ini.

Tujuan

Tujuan dari pengerjaan kasus ini adalah memberikan pemahaman yang utuh tentang bagaimana empat algoritma pencarian bekerja pada satu peta yang sama sekaligus melatih kemampuan dalam menerjemahkan teori ke dalam program yang dapat diuji. Melalui implementasi UCS, IDS, GBFS, dan A Star dalam sebuah program sederhana,

mahasiswa diharapkan mampu mencatat secara tertib urutan simpul yang diperluas, rute yang ditemukan dari GU ke LK, serta total biaya perjalanan yang dihasilkan oleh masing masing metode. Perbandingan tersebut kemudian dianalisis untuk menjawab pertanyaan yang lebih dalam yaitu mengapa suatu algoritma bisa menemukan jalur yang lebih murah, mengapa ada algoritma yang langkahnya sedikit tetapi hasilnya belum tentu optimal, serta bagaimana peran bobot jalur dan nilai heuristik dalam membentuk keputusan robot. Pada akhirnya laporan yang dikumpulkan dalam bentuk PDF tidak hanya menampilkan tangkapan luaran program sebagai bukti, tetapi juga memuat argumentasi yang menjelaskan kelebihan dan kekurangan setiap metode sehingga kelompok mampu memberikan rekomendasi algoritma yang paling tepat untuk kebutuhan robot kurir kampus.

Metode yang Digunakan

Keempat algoritma dijalankan pada graf dan heuristik yang sama agar adil. Aturan bersama adalah tie break alfabetis: bila dua simpul memiliki prioritas sama maka nama yang lebih kecil menurut abjad dipilih dulu.

3.1 UCS (Uniform Cost Search)

UCS selalu memperluas simpul dengan biaya perjalanan $g(n)$ terkecil dari titik awal GU. Ia dijadikan acuan kebenaran karena selama bobot positif maka rute yang pertama mencapai LK dijamin termurah, walau ekspansinya banyak.

3.2 IDS (Iterative Deepening Search)

IDS menaikkan batas kedalaman bertahap mulai dari nol lalu satu lalu dua dan seterusnya. Tiap batas dijalankan DFS alfabetis dengan path checking agar tidak berputar. IDS memakai jumlah edge sebagai kedalaman dan bukan bobot, sedangkan bobot baru dipakai di akhir untuk total biaya, sehingga ia dapat menemukan rute yang dangkal tetapi mahal.

3.3 GBFS (Greedy Best First Search)

GBFS selalu memperluas simpul dengan heuristik $h(n)$ terkecil tanpa peduli biaya yang sudah dikeluarkan. Ia cepat dan sedikit ekspansi, tetapi rapuh dan tidak menjamin optimal.

3.4 A Star

A Star memakai $f(n) = g(n) + h(n)$, gabungan biaya pasti dan perkiraan sisa. Karena h admissible dan konsisten di kasus ini, A Star optimal seperti UCS tetapi lebih sedikit ekspansi karena memangkas cabang mahal sejak dini.

Hasil Luaran Program

Perintah: `python search_kampus.py`. Salinan persis luarannya:

```
[UCS] ekspansi: GU -> PB -> R -> GKU -> PR -> K -> M -> IF -> A -> AS -> LK
g: GU=0 PB=3 R=4 GKU=7 PR=7 K=9 M=9 IF=11 A=12 AS=14 LK=14
Rute: GU -> R -> PR -> IF -> LK | Cost 14 (11 ekspansi)
[IDS] limit0: GU (belum) | limit1: GU->PB->R (belum)
limit2: GU->PB->GKU->K->R->M->PR (belum, 7 kunjungan)
limit3: 17 kunjungan (belum)
limit4: GU->PB->GKU->IF->AS->LK => KETEMU GU->PB->GKU->IF->LK
Cost 17, depth 4, total kumulatif 34
[GBFS] ekspansi: GU->R->PR->IF->LK | Cost 14 (5 ekspansi)
[A*] ekspansi: GU->PB->R->PR->IF->GKU->LK
(g,h,f): GU(0,12,12) PB(3,10,13) R(4,9,13) PR(7,6,13) IF(11,2,13) GKU(7,7,14) LK(14,0,14)
Rute: GU->R->PR->IF->LK | Cost 14 (7 ekspansi)
```

Brute force semua jalur sederhana: termurah GU R PR IF LK = $4+3+4+3 = 14$, runner-up GU PB GKU PR IF LK = $3+4+2+4+3 = 16$. Jadi 14 optimal, 17 (IDS) suboptimal.

Catatan revisi jalur 16. IDS memakai batas depth dan bukan batas biaya. Jalur biaya 16 berdepth 5 sedangkan rute IDS berdepth 4, sehingga saat limit 4 menemukan solusi pertama di cabang PB maka pencarian berhenti dan tidak pernah mencoba jalur depth 5. Jadi $16 < 17$ dari sisi biaya tetapi $5 > 4$ dari sisi depth. Varian sadar biaya dari IDS adalah IDA Star dengan batas $f = g + h$.

Jalur	Hitung	Cost	Depth	Keterangan
GU R PR IF LK	$4+3+4+3$	14	4	Optimal (UCS, GBFS, A Star)
GU PB GKU PR IF LK	$3+4+2+4+3$	16	5	Lebih murah dari IDS, depth 5
GU PB GKU IF LK	$3+4+7+3$	17	4	Rute IDS, depth 4 cabang PB
GU R M A LK	$4+5+3+5$	17	4	Contoh depth 4 lain

Analisis Perbandingan Metode

Aspek	UCS	IDS	GBFS	A Star
Prioritas	$g(n)$	depth via DFS	$h(n)$	$f = g + h$
Ekspansi	11	34 kumulatif	5	7
Rute	GU R PR IF LK	GU PB GKU IF LK	GU R PR IF LK	GU R PR IF LK
Cost	14 optimal	17 suboptimal	14 kebetulan	14 optimal
Edge	4	4	4	4

Urutan ekspansi. UCS: GU(0), PB(3), R(4), GKU(7), PR(7), K(9), M(9), IF(11), A(12), AS(14), LK(14), total 11 simpul. A Star: GU(12), PB(13), R(13), PR(13), IF(13), GKU(14), LK(14), hanya 7 simpul karena memangkas K, M, A, AS, SC. GBFS: GU, R, PR, IF, LK, hanya 5 simpul mengikuti h menurun 12, 9, 6, 2, 0.

Rute dan biaya. UCS dan A Star optimal 14. GBFS ikut 14 tetapi itu kebetulan karena heuristik mengarah ke koridor optimal, dan teori tidak menjaminkannya. IDS mendapat 17 pada depth 4 karena berhenti di solusi depth 4 pertama cabang PB yang diperiksa sebelum cabang R. Bobot tidak mengubah urutan DFS IDS dan hanya dipakai menghitung biaya akhir.

Efisiensi. IDS paling boros: $1+3+7+17+6 = 34$ kunjungan untuk 12 simpul. Hemat memori $O(d)$ dan lengkap, tetapi buta biaya dan hanya cocok bila bobot seragam, sedangkan di sini bobot 2 sampai 7.

Peran heuristik. Karena h admissible dan konsisten, A Star optimal sekaligus efisien (7 vs 11). GBFS tercepat (5) tetapi rapuh bila ada overestimate. UCS aman tanpa heuristik tetapi membayar ekspansi ekstra.

Kesimpulan

A Star adalah algoritma yang paling tepat untuk robot kurir pada peta ini karena menemukan rute optimal biaya 14 dengan hanya 7 ekspansi. UCS menjadi pembanding kebenaran dengan biaya yang sama tetapi 11 ekspansi. GBFS cepat (5 ekspansi) namun tidak terjamin. IDS tidak disarankan untuk optimasi biaya karena menghasilkan 17 dengan 34 kunjungan, dan hanya relevan bila memori sangat terbatas atau yang dipentingkan adalah jumlah transit dan bukan jarak. Rekomendasi praktis: pakai A Star sebagai algoritma utama dan UCS sebagai acuan pengujian setiap kali peta atau heuristik diubah.

Daftar Pustaka

Russell, S. dan Norvig, P. (2021). *Artificial Intelligence: A Modern Approach*, edisi keempat. Hoboken: Pearson. Bab 3 dan 4 untuk teori UCS, IDS, GBFS, dan A Star.

Materi kuliah Inteligensi Buatan, Pertemuan 3, Tugas Kelompok Berbasis Kasus 01 (2026). Spesifikasi graf GU ke LK, nilai $h(n)$, dan ketentuan luaran program serta analisis.

Dokumentasi Python 3. Modul *heapq*: priority queue untuk antrean UCS, GBFS, dan A Star.

Hart, P., Nilsson, N., dan Raphael, B. (1968). A Formal Basis for the Heuristic Determination of Minimum Cost Paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2), 100-107. Dasar sifat admissible dan konsisten pada A Star.

Lampiran (Source Code)

File **search_kampus.py** lengkap di bawah. Reproduksi: simpan bersama dokumen ini lalu jalankan **python search_kampus.py**. Perintah **python buat_docx.py** membuat ulang DOCX ini dan **python buat_laporan.py** membuat ulang PDF (butuh reportlab).

```

"""
Tugas Kelompok - Berbasis Kasus 01
Robot kurir kampus: GU (Gerbang Utama) -> LK (Lab Komputasi)
Metode: UCS, IDS, GBFS, A*

Graf diverifikasi dari gambar hires (19 edge, undirected).
Tie-break: alfabetis agar deterministik.
"""

import heapq
from collections import defaultdict

# ----- Graf -----
GRAPH = {
    "GU": [("PB", 3), ("R", 4)],
    "PB": [("GU", 3), ("GKU", 4), ("K", 6)],
    "R": [("GU", 4), ("PR", 3), ("M", 5)],
    "GKU": [("PB", 4), ("PR", 2), ("K", 3), ("IF", 7)],
    "K": [("PB", 6), ("GKU", 3), ("AS", 5)],
    "PR": [("R", 3), ("GKU", 2), ("A", 6), ("IF", 4)],

```

```

"A": [("PR", 6), ("M", 3), ("LK", 5)],
"M": [("R", 5), ("A", 3), ("SC", 6)],
"IF": [("GKU", 7), ("PR", 4), ("AS", 4), ("LK", 3)],
"AS": [("K", 5), ("IF", 4), ("SC", 6)],
"SC": [("M", 6), ("AS", 6), ("LK", 4)],
"LK": [("A", 5), ("IF", 3), ("SC", 4)],
}

H = {
    "GU": 12, "PB": 10, "R": 9, "GKU": 7, "PR": 6,
    "K": 9, "M": 7, "A": 4, "AS": 5, "IF": 2, "SC": 3, "LK": 0,
}

NAMA = {
    "GU": "Gerbang Utama", "PB": "Parkir Barat", "R": "Rektorat",
    "GKU": "Gedung Kuliah Umum", "PR": "Perpustakaan", "K": "Kantin",
    "M": "Masjid", "A": "Aula", "AS": "Asrama", "IF": "Teknik Informatika",
    "SC": "Sport Center", "LK": "Lab Komputasi",
}

START, GOAL = "GU", "LK"

def neighbors_sorted(node):
    return sorted(GRAPH[node], key=lambda x: x[0])

def reconstruct(parent, goal):
    path, cur = [goal], goal
    while parent[cur] is not None:
        cur = parent[cur]
        path.append(cur)
    return list(reversed(path))

def path_cost(path):
    total = 0
    for a, b in zip(path, path[1:]):
        for nb, w in GRAPH[a]:
            if nb == b:
                total += w
                break
    return total

# ----- UCS -----
def ucs(start=START, goal=GOAL):
    pq = [(0, start)] # (g, node)
    best = {start: 0}
    parent = {start: None}
    expansion = [] # urutan pop
    pop_g = {}
    visited = set()
    while pq:
        g, n = heapq.heappop(pq)
        if n in visited:
            continue
        # jika ada jalur lebih baik sudah tercatat, skip basi
        if g > best.get(n, float("inf")):
            continue
        visited.add(n)
        expansion.append(n)
        pop_g[n] = g
        if n == goal:
            break
        for nb, w in neighbors_sorted(n):
            ng = g + w
            if nb not in best or ng < best[nb]:
                # izinkan reopen bila lebih baik (penting utk A*, utk UCS sama saja)
                if nb not in visited or ng < best[nb]:
                    best[nb] = ng
                    parent[nb] = n
                    heapq.heappush(pq, (ng, nb))
    path = reconstruct(parent, goal) if goal in parent else []
    return {"expansion": expansion, "path": path,
            "cost": path_cost(path) if path else None, "g": pop_g}

# ----- GBFS -----
def gbfs(start=START, goal=GOAL):
    pq = [(H[start], start)] # (h, node)
    parent = {start: None}
    expansion = []
    visited = set()
    g_cost = {start: 0}
    while pq:
        _, n = heapq.heappop(pq)
        if n in visited:
            continue
        visited.add(n)
        expansion.append(n)
        if n == goal:
            break

```

```

        for nb, w in neighbors_sorted(n):
            if nb not in visited and nb not in [x[1] for x in pq]:
                parent[nb] = n
                g_cost[nb] = g_cost[n] + w
                heapq.heappush(pq, (H[nb], nb))
            elif nb in g_cost and g_cost[n] + w < g_cost[nb]:
                # update bila jalur lebih murah meski h sama (variasi umum)
                pass
    path = reconstruct(parent, goal) if goal in parent else []
    return {"expansion": expansion, "path": path,
            "cost": path_cost(path) if path else None}

# ----- A* -----
def astar(start=START, goal=GOAL):
    pq = [(H[start], 0, start)] # (f, g, node)
    best_g = {start: 0}
    parent = {start: None}
    expansion = []
    pop_f = {}
    visited_closed = set()
    while pq:
        f, g, n = heapq.heappop(pq)
        if n in visited_closed and g >= best_g.get(n, float("inf")):
            continue
        if g > best_g.get(n, float("inf")):
            continue
        visited_closed.add(n)
        expansion.append(n)
        pop_f[n] = (g, H[n], f)
        if n == goal:
            break
        for nb, w in neighbors_sorted(n):
            ng = g + w
            if nb not in best_g or ng < best_g[nb]:
                best_g[nb] = ng
                parent[nb] = n
                heapq.heappush(pq, (ng + H[nb], ng, nb))
            elif nb not in visited_closed:
                pass
    path = reconstruct(parent, goal) if goal in parent else []
    return {"expansion": expansion, "path": path,
            "cost": path_cost(path) if path else None, "detail": pop_f}

# ----- IDS -----
def dls(limit, start=START, goal=GOAL, log=None):
    """Depth-Limited DFS (path-checking, alfabetis). Return path atau None."""
    found = None
    def rec(n, depth, path, path_set):
        nonlocal found
        if log is not None:
            log.append(n)
        if found is not None:
            return True
        if n == goal:
            found = list(path)
            return True
        if depth == limit:
            return False
        for nb, _ in neighbors_sorted(n):
            if nb in path_set: # hindari cycle dalam satu jalur
                continue
            path.append(nb)
            path_set.add(nb)
            if rec(nb, depth + 1, path, path_set):
                return True
            path.pop()
            path_set.remove(nb)
        return False
    rec(start, 0, [start], {start})
    return found

def ids(start=START, goal=GOAL, max_depth=10):
    per_limit = {}
    total_expansions = 0
    solution = None
    solution_depth = None
    for lim in range(0, max_depth + 1):
        log = []
        p = dls(lim, start, goal, log)
        per_limit[lim] = {"visited_order": list(log), "count": len(log),
                        "found": p}
        total_expansions += len(log)
        if p is not None:
            solution = p
            solution_depth = lim
            break
    cost = path_cost(solution) if solution else None
    return {"per_limit": per_limit, "path": solution,
            "cost": cost, "depth": solution_depth,
            "total_expanded": total_expansions,

```

```

        "expansion": per_limit[solution_depth]["visited_order"] if solution else []}

# ----- main -----
def fmt_path(p):
    return " -> ".join(p) if p else "-"

def main():
    u = ucs()
    i = ids()
    g = gbfs()
    a = astar()

    print("=" * 70)
    print("KASUS 01: Robot kurir kampus GU -> LK")
    print("=" * 70)

    print("\n[UCS] Urutan ekspansi (pop terkecil g):")
    print(" ", " -> ".join(u["expansion"]))
    print(" g saat pop:", {n: u["g"][n] for n in u["expansion"]})
    print(" Rute :", fmt_path(u["path"]))
    print(" Cost :", u["cost"], "| node diekspansi:", len(u["expansion"]))

    print("\n[IDS] Iterative Deepening (DFS alfabetis, limit 0..n):")
    for lim, d in i["per_limit"].items():
        status = "KETEMU " + fmt_path(d["found"]) if d["found"] else "belum ketemu"
        print(f" limit={lim} ({d['count']} kunjungan): {' -> '.join(d['visited_order'])} => {
status}")
        print(" Rute :", fmt_path(i["path"]))
        print(" Cost (jumlah bobot) :", i["cost"], "| depth solusi:", i["depth"],
            "| total kunjungan kumulatif:", i["total_expanded"])

    print("\n[GBFS] Urutan ekspansi (pop terkecil h):")
    print(" ", " -> ".join(g["expansion"]))
    print(" Rute :", fmt_path(g["path"]))
    print(" Cost :", g["cost"], "| node diekspansi:", len(g["expansion"]))

    print("\n[A*] Urutan ekspansi (pop terkecil f=g+h):")
    print(" ", " -> ".join(a["expansion"]))
    print(" detail pop (g,h,f):", {n: {"g": v[0], "h": v[1], "f": v[2]} for n, v in a[
"detail"].items()})
    print(" Rute :", fmt_path(a["path"]))
    print(" Cost :", a["cost"], "| node diekspansi:", len(a["expansion"]))

    print("\n--- RINGKASAN ---")
    for nama, r in [("UCS", u), ("IDS", i), ("GBFS", g), ("A*", a)]:
        n_exp = len(r["expansion"]) if nama != "IDS" else f"{i['total_expanded']}" (kumulatif,
solusi depth {i['depth']})"
        print(f"{nama:5s} | ekspansi: {n_exp} | rute: {fmt_path(r['path'])} | cost: {r[
'cost']}")

if __name__ == "__main__":
    main()

```

Catatan: ganti placeholder Rx dan nim1 nim2 nim3 dengan data kelompok, lalu tambahkan screenshot terminal dan gambar graf sebelum dikumpulkan.